Operating Systems Interview Guide

85 Most Asked Questions & Answers

For B.Tech CS / AI-DS · Campus Placements · 2026

Covers everything asked across HR, technical, and coding rounds — OS basics & kernel structure, process management, CPU scheduling, synchronization & deadlocks, memory management (paging, segmentation, virtual memory), file systems & disk scheduling, and scenario-based questions — with diagrams where it helps.

Table of Contents

OS Basics & Structure

11 Questions

Q1 What is an Operating System? What are its main functions?

An Operating System (OS) is system software that acts as an intermediary between computer hardware and the user/applications, managing hardware resources and providing common services. Main functions: process management, memory management, file system management, I/O device management, security/access control, and providing a user interface (CLI/GUI).

Q2 What are the main objectives of an Operating System?

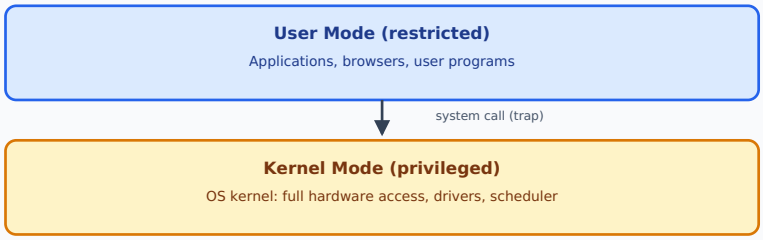
- **Convenience:** make the computer easier to use for end users.
- **Efficiency:** manage hardware resources (CPU, memory, I/O) efficiently.
- **Ability to evolve:** allow new features/hardware to be added without disrupting existing services.

Q3 What is the difference between a Kernel and an Operating System?

The **kernel** is the core component of the OS that directly manages hardware resources (CPU scheduling, memory management, device drivers) and runs in privileged mode. The **Operating System** is the complete software package, which includes the kernel plus additional utilities, libraries, system programs, and user interfaces built around it.

Q4 What is the difference between Kernel mode and User mode?

Kernel mode (privileged/supervisor mode): the CPU can execute any instruction and access all hardware/memory directly — used by the OS kernel. **User mode:** a restricted mode where applications run with limited privileges, preventing them from directly accessing hardware or critical memory, improving system stability and security. A 'mode bit' in hardware tracks the current mode.



Q5 What is a system call? Why is it needed?

A system call is the mechanism through which a user-mode program requests a service from the kernel (e.g., file I/O, process creation, memory allocation) — since user programs can't directly execute privileged operations, they must 'trap' into kernel mode via a system call, which validates the request and performs it on the program's behalf before returning control.

Q6 What are the different types of Operating Systems?

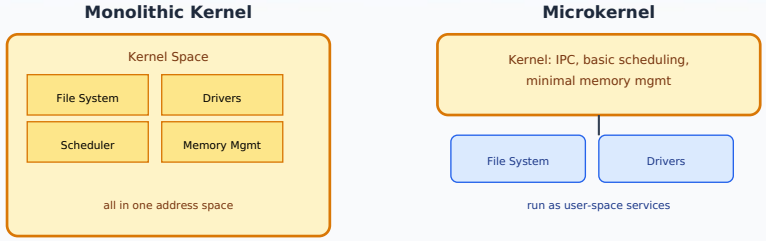
- **Batch OS:** executes jobs in batches with no user interaction.
- **Time-sharing/Multitasking OS:** allows multiple users/processes to share CPU time via rapid switching.
- **Distributed OS:** manages a group of independent computers, making them appear as a single system.
- **Real-Time OS (RTOS):** guarantees processing within strict, predictable time constraints (hard/soft real-time).
- **Network OS:** manages data, users, and security across a network.

Q7 What is the difference between multiprogramming, multitasking, and multiprocessing?

- **Multiprogramming:** multiple programs reside in memory at once, and the CPU switches between them to keep it busy (maximizing CPU utilization), but not necessarily quickly enough to feel interactive.
- **Multitasking:** an extension of multiprogramming with fast enough context switching that users perceive multiple programs running 'simultaneously' (time-sharing).
- **Multiprocessing:** a system with multiple physical CPUs/cores, allowing true parallel execution of processes.

Q8 What is the difference between monolithic and microkernel architecture?

A **monolithic kernel** runs the entire OS (process management, memory management, device drivers, file systems) as a single large program in kernel mode — faster (fewer context switches) but a bug in any component can crash the whole system. A **microkernel** keeps only the most essential functions (basic IPC, minimal scheduling, memory management) in kernel mode, running other services (drivers, file systems) as user-mode processes — more modular/stable/secure but slower due to increased inter-process communication overhead.



Q9 What is virtualization in the context of operating systems?

Virtualization allows multiple isolated virtual instances (e.g., virtual machines, each with its own OS) to run on top of a single physical machine, managed by a hypervisor that allocates and abstracts the underlying physical hardware resources among the different virtual instances.

Q10 What is a Real-Time Operating System (RTOS)? What are its types?

An RTOS processes inputs and produces outputs within strictly guaranteed, predictable time constraints. **Hard real-time** systems must never miss a deadline (e.g., pacemakers, aircraft control) — missing one is a system failure. **Soft real-time** systems tolerate occasional missed deadlines with some degradation in quality (e.g., video streaming).

Q11 What is the boot process of an Operating System (brief)?

On power-on: the BIOS/UEFI firmware runs POST (Power-On Self-Test) checks, then locates and loads the bootloader from a boot device, which in turn loads the OS kernel into memory and transfers control to it; the kernel then initializes device drivers, mounts the root filesystem, and starts essential system processes/services.

Process Management

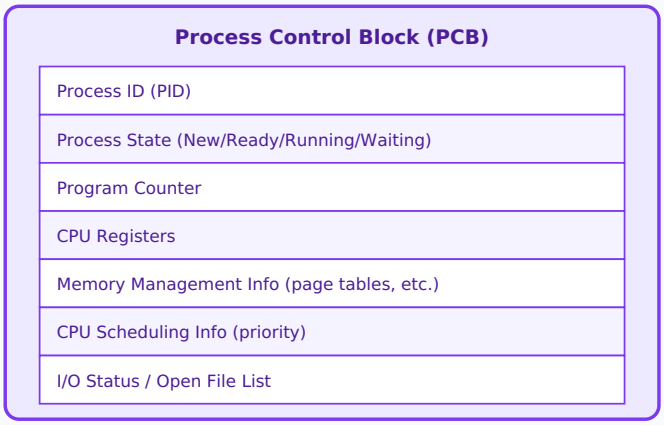
12 Questions

Q12 What is a process? How is it different from a program?

A **program** is a passive, static set of instructions stored on disk (an executable file). A **process** is an active, dynamic instance of a program currently being executed, with its own allocated memory (code, data, stack, heap), CPU register values, and execution state — the same program can be run as multiple independent processes.

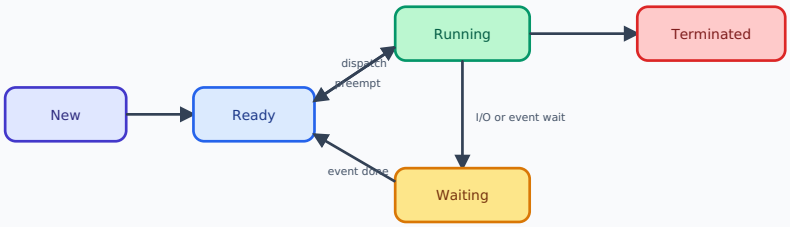
Q13 What is a Process Control Block (PCB)?

A PCB is a data structure maintained by the OS for each process, storing all information needed to manage and resume it: process ID, process state, program counter, CPU register values, memory management info, CPU scheduling info (priority), and I/O status/open file information.



Q14 What are the different states of a process?

New (being created) → **Ready** (waiting for CPU allocation) → **Running** (currently executing on CPU) → **Waiting/Blocked** (waiting for an I/O or event) → **Terminated** (finished execution). A running process can move back to Ready (preempted) or to Waiting (I/O request), and from Waiting back to Ready once the event completes.



Q15 What is context switching? Why does it have overhead?

Context switching is the process of saving the state (PCB: registers, program counter, memory maps) of a currently running process and loading the saved state of another process, so the CPU can switch to executing it. It has overhead because saving/restoring register/memory state, flushing CPU caches/TLB, and the switching logic itself all consume CPU cycles without doing any useful application work.

Q16 What is the difference between a process and a thread?

A **process** is an independent execution unit with its own separate memory address space (isolated from other processes). A **thread** is a lightweight unit of execution within a process, sharing the process's memory space (code, data, heap) with other threads of the same process, but maintaining its own stack and register set — making thread creation/context switching cheaper than process creation/switching.

Q17 What is the difference between a user-level thread and a kernel-level thread?

User-level threads are managed entirely by a user-space thread library without kernel involvement — fast to create/switch but the kernel isn't aware of them (if one blocks on I/O, the whole process may block). **Kernel-level threads** are managed and scheduled directly by the OS kernel — slightly higher overhead for creation/switching, but the kernel can schedule them independently across multiple CPUs and one thread blocking doesn't block the others.

Q18 **What are the different multithreading models?**

- **Many-to-One:** many user threads mapped to one kernel thread — fast but no true parallelism, one block stalls all.
- **One-to-One:** each user thread maps to its own kernel thread — true parallelism, but thread creation has kernel overhead.
- **Many-to-Many:** many user threads multiplexed onto a smaller or equal number of kernel threads — balances flexibility and true concurrency.

Q19 **What is an orphan process and a zombie process?**

An **orphan process** is a still-running child process whose parent has terminated before it — typically adopted by the 'init'/system process. A **zombie process** is a process that has finished execution but still has an entry in the process table because its parent hasn't yet read its exit status (via wait()) — it consumes a PID slot but no other resources until reaped.

Q20 **What is the difference between a foreground and background process?**

A **foreground process** requires direct interaction with the user and occupies the terminal/interface until it completes. A **background process** runs independently without blocking the user's interaction with the terminal/shell, typically launched with '&' in Unix-like systems.

Q21 **What is inter-process communication (IPC)? Name common mechanisms.**

IPC refers to mechanisms that allow separate processes to communicate and synchronize with each other, since they don't share memory by default. Common mechanisms: **Pipes** (unidirectional byte stream between related processes), **Message Queues**, **Shared Memory** (fastest, direct shared memory segment, requires synchronization), **Sockets** (network/cross-machine communication), and **Signals** (simple event notifications).

Q22 **What is the difference between shared memory and message passing for IPC?**

Shared memory lets processes directly read/write a common memory region — very fast since no kernel involvement is needed for each data transfer, but requires explicit synchronization to avoid race conditions. **Message passing** involves the kernel copying data between processes via send/receive system calls — slower (kernel-mediated, extra copying) but simpler to use correctly and naturally synchronized (avoids shared-state races).

Q23 **What is a daemon process?**

A daemon is a background process that runs continuously, typically starting at system boot, without direct user interaction, providing ongoing services (e.g., a web server, print spooler, or cron scheduler) until the system shuts down or the service is explicitly stopped.

CPU Scheduling

10 Questions

Q24 What is CPU scheduling? Why is it needed?

CPU scheduling is the process by which the OS decides which ready process gets to use the CPU next, aiming to maximize CPU utilization, throughput, and fairness while minimizing waiting time, response time, and turnaround time — necessary because there are typically many more ready processes than available CPUs.

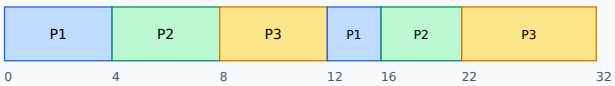
Q25 What is the difference between preemptive and non-preemptive scheduling?

In **preemptive scheduling**, the OS can forcibly take the CPU away from a running process (e.g., when a higher-priority process arrives or a time slice expires) before it voluntarily gives it up. In **non-preemptive scheduling**, once a process is given the CPU, it keeps it until it either finishes or voluntarily blocks (e.g., for I/O) — the OS cannot interrupt it.

Q26 What are the common CPU scheduling algorithms?

- **FCFS (First-Come, First-Served):** non-preemptive, processes run in arrival order.
- **SJF (Shortest Job First):** runs the process with the shortest burst time next; can be preemptive (SRTF) or non-preemptive.
- **Priority Scheduling:** runs the highest-priority process first; can be preemptive or non-preemptive.
- **Round Robin (RR):** each process gets a fixed time quantum in circular order — preemptive, fair, common in time-sharing systems.
- **Multilevel Queue / Multilevel Feedback Queue:** separates processes into different priority queues with different scheduling rules, allowing processes to move between queues based on behavior.

Example Gantt Chart (Round Robin, quantum=4)



Each process gets a fixed time slice; remaining burst is requeued if not finished

Q27 What is turnaround time, waiting time, and response time in scheduling?

- **Turnaround time** = Completion time − Arrival time (total time from arrival to finishing).
- **Waiting time** = Turnaround time − Burst time (time spent waiting in the ready queue).
- **Response time** = Time of first CPU response − Arrival time (time until the process first gets the CPU, important for interactive systems).

Q28 What is the convoy effect in CPU scheduling?

The convoy effect occurs in FCFS scheduling when a long CPU-bound process holds the CPU for a long time, causing many shorter processes behind it in the queue to wait unnecessarily — significantly increasing average waiting time, similar to slow traffic behind a large vehicle on a highway.

Q29 What is starvation in scheduling? How is it typically resolved?

Starvation occurs when a process is perpetually denied CPU time because the scheduler continuously favors other (e.g., higher-priority) processes — it's typically resolved using **aging**, where a waiting process's priority is gradually increased the longer it waits, eventually guaranteeing it gets scheduled.

Q30 What is the difference between SJF and SRTF scheduling?

SJF (Shortest Job First) is generally non-preemptive — once a process starts, it runs to completion even if a shorter job arrives. **SRTF (Shortest Remaining Time First)** is the preemptive version — if a new process arrives with a burst time shorter than the currently running process's remaining time, the CPU is immediately switched to the new (shorter) process.

Q31 How do you choose an optimal time quantum in Round Robin scheduling?

The time quantum should be large enough that most processes can complete their CPU burst within it (minimizing context-switch overhead), but small enough to keep response times reasonable for interactive use. Too small a quantum causes excessive context-switching overhead; too large a quantum makes Round Robin behave like FCFS. A common rule of thumb is that 80% of CPU bursts should be shorter than the time quantum.

Q32 What is a Multilevel Feedback Queue scheduler?

It's an advanced scheduling scheme using multiple queues with different priority levels and possibly different scheduling algorithms/time quanta per queue. Processes can move between queues based on their behavior — e.g., a CPU-bound process that uses its full time quantum repeatedly gets demoted to a lower-priority queue, while an I/O-bound process that frequently gives up the CPU early may be promoted — allowing the scheduler to adapt dynamically without prior knowledge of burst times.

Q33 **What is Highest Response Ratio Next (HRRN) scheduling?**

HRRN is a non-preemptive scheduling algorithm that selects the process with the highest 'response ratio,' calculated as (Waiting Time + Burst Time) / Burst Time. It effectively balances SJF's efficiency for short jobs with fairness for long jobs, since a long-waiting process's ratio increases over time, preventing indefinite starvation.

Process Synchronization & Deadlocks

12 Questions

Q34 What is a race condition?

A race condition occurs when multiple processes/threads access and manipulate shared data concurrently, and the final outcome depends on the unpredictable timing/order of their execution — typically leading to incorrect or inconsistent results without proper synchronization.

Q35 What is the critical section problem?

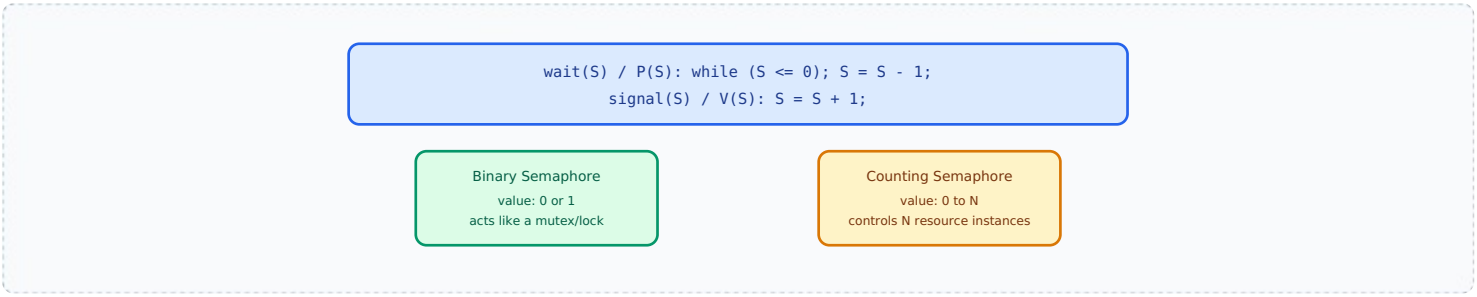
The critical section is the part of a program where shared resources are accessed/modified. The critical section problem is designing a protocol ensuring that when one process is executing in its critical section, no other process is allowed to execute in its own critical section for the same resource — preventing race conditions.

Q36 What are the three requirements a solution to the critical section problem must satisfy?

- **Mutual Exclusion:** only one process can be in its critical section at a time.
- **Progress:** if no process is in its critical section, a process wanting to enter should not be indefinitely delayed by processes not interested in entering.
- **Bounded Waiting:** there must be a limit on how many times other processes can enter their critical section after a process has requested entry, before that request is granted.

Q37 What is a semaphore? What are its types?

A semaphore is an integer variable used for process synchronization, accessed only through two atomic operations: **wait()/P()** (decrements, blocks if the value would go negative) and **signal()/V()** (increments, potentially waking a blocked process). Types: **Binary semaphore** (value 0 or 1, acts like a lock/mutex) and **Counting semaphore** (value can range over a larger domain, used to control access to a resource with multiple instances).



Q38 What is the difference between a mutex and a semaphore?

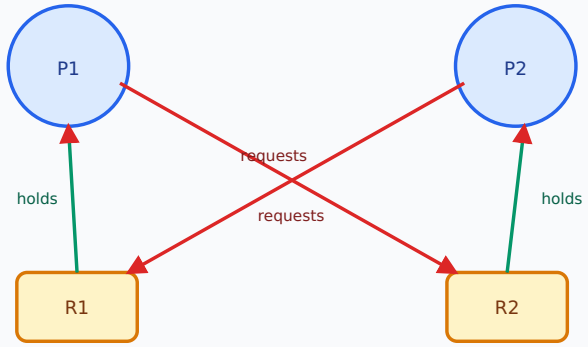
A **mutex** is a locking mechanism providing strict mutual exclusion — it can only be released by the same thread/process that acquired it, and has ownership semantics. A **semaphore** is a more general signaling mechanism (a counter) that can be incremented/decremented by any process, doesn't have strict ownership, and a counting semaphore can allow multiple processes to access a resource pool simultaneously (up to its count) — used for signaling between threads, not just exclusion.

Q39 What is a monitor in process synchronization?

A monitor is a high-level synchronization construct that encapsulates shared data along with the procedures that operate on it, automatically ensuring only one process can execute inside the monitor's procedures at a time (implicit mutual exclusion). Monitors use condition variables (with wait()/signal() operations) to allow processes to block/resume based on specific conditions, offering a more structured, less error-prone alternative to raw semaphores.

Q40 What is a deadlock? What are the four necessary conditions for it?

A deadlock is a state where a set of processes are all blocked, each waiting for a resource held by another process in the same set, so none can proceed. Four necessary conditions (Coffman conditions), all of which must hold: **Mutual Exclusion** (resources aren't shareable), **Hold and Wait** (a process holds a resource while waiting for another), **No Preemption** (resources can't be forcibly taken away), and **Circular Wait** (a cycle of processes each waiting for a resource held by the next).



Circular wait: P1 holds R1, wants R2; P2 holds R2, wants R1 → deadlock

Q41 What are the different strategies to handle deadlocks?

- **Deadlock Prevention:** design the system to ensure at least one of the four necessary conditions can never hold.
- **Deadlock Avoidance:** dynamically check if granting a resource request could lead to an unsafe state (e.g., Banker's Algorithm) before granting it.
- **Deadlock Detection and Recovery:** allow deadlocks to occur, periodically check for them (via resource allocation graphs), and recover by terminating processes or preempting resources.
- **Ignore the problem (Ostrich algorithm):** assume deadlocks are rare enough not to be worth the overhead of handling — used by some general-purpose OSes like classic UNIX.

Q42 What is the Banker's Algorithm?

The Banker's Algorithm is a deadlock-avoidance algorithm that, before granting a resource request, simulates the allocation and checks if the resulting system state would still be 'safe' — meaning there exists at least one sequence in which all processes could still complete without deadlock. If the resulting state would be unsafe, the request is denied/postponed, even if resources are technically available.

Q43 What is a safe state vs an unsafe state in the context of deadlock avoidance?

A **safe state** is one where the system can allocate resources to each process in some order and guarantee all processes can eventually complete without entering a deadlock. An **unsafe state** doesn't necessarily mean deadlock has occurred, but there's no guarantee all processes can complete — a deadlock becomes possible depending on future requests.

Q44 What is the difference between deadlock and starvation?

In a **deadlock**, involved processes are permanently blocked in a circular wait — none can ever proceed. In **starvation**, a process is repeatedly denied a resource it needs (e.g., always losing out to higher-priority requests), but it's not stuck in a cycle — the resource is available in principle, just never actually granted to it.

Q45 What is a resource allocation graph (RAG)? How is it used to detect deadlock?

A RAG is a directed graph representing resource allocation state: process nodes and resource nodes, with a request edge (process → resource) indicating a process is waiting for a resource, and an assignment edge (resource → process) indicating a resource is held by a process. If the graph contains a cycle (and each resource type has only one instance), a deadlock exists; with multiple instances per resource type, a cycle is necessary but not always sufficient for deadlock.

Memory Management

16 Questions

Q46 What is the purpose of memory management in an OS?

Memory management handles the allocation and deallocation of main memory (RAM) to processes, keeps track of which parts of memory are in use, protects processes from accessing each other's memory, and enables techniques like virtual memory to run programs larger than physical memory allows.

Q47 What is the difference between logical address and physical address?

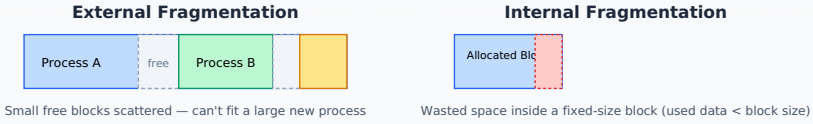
A **logical (virtual) address** is generated by the CPU during program execution and is relative to the process's own address space. A **physical address** is the actual location in main memory (RAM). The Memory Management Unit (MMU) translates logical addresses to physical addresses at runtime.

Q48 What is the difference between contiguous and non-contiguous memory allocation?

In **contiguous allocation**, each process is allocated a single continuous block of memory. In **non-contiguous allocation** (e.g., paging, segmentation), a process's memory can be split into multiple, non-adjacent blocks scattered across physical memory, offering more flexibility and reducing fragmentation issues.

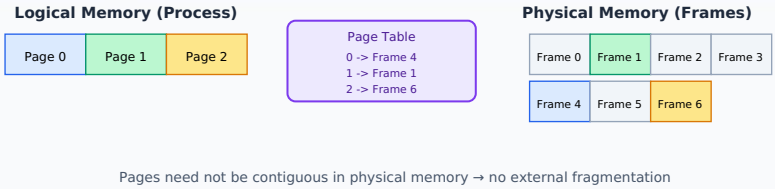
Q49 What is fragmentation? What is the difference between internal and external fragmentation?

Fragmentation is wasted memory that cannot be effectively used. **Internal fragmentation** occurs when allocated memory is slightly larger than requested, wasting the leftover space within an allocated block (common with fixed-size partitioning/paging). **External fragmentation** occurs when free memory is split into many small, non-contiguous blocks, none large enough individually to satisfy a request, even though the total free memory is sufficient (common with variable-size/contiguous allocation).



Q50 What is paging? How does it solve external fragmentation?

Paging divides both physical memory into fixed-size blocks called **frames** and a process's logical memory into equal-sized blocks called **pages**. A page table maps each page to a frame, allowing a process's pages to be scattered non-contiguously anywhere in physical memory — this eliminates external fragmentation entirely (though it can still have internal fragmentation in the last, partially-filled page).



Q51 What is a page table? What is a page table entry?

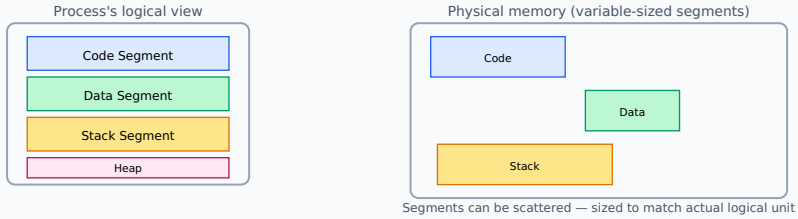
A page table is a per-process data structure maintained by the OS that maps each virtual page number to its corresponding physical frame number. Each page table entry (PTE) typically also contains control bits like a valid/invalid bit, protection bits (read/write/execute), and a dirty/modified bit.

Q52 What is a TLB (Translation Lookaside Buffer)? Why is it used?

A TLB is a small, fast hardware cache within the CPU that stores recent virtual-to-physical address translations. Since consulting the full page table in main memory for every memory access would be slow, the TLB provides much faster lookups for recently used translations — a 'TLB hit' avoids the extra memory access, while a 'TLB miss' requires the slower page table walk.

Q53 What is segmentation? How is it different from paging?

Segmentation divides a process's memory into logically meaningful, variable-sized segments (e.g., code, stack, heap, data), each with its own base and limit, matching how programmers naturally think about a program's structure — unlike paging's fixed-size, purely physical blocks that ignore logical program structure. Segmentation can suffer from external fragmentation (since segments are variable-sized), which paging avoids.



Q54 What is virtual memory? Why is it useful?

Virtual memory is a technique that gives each process the illusion of having its own large, contiguous address space, even if it's larger than the actual physical RAM available, by using disk storage (swap space) as an extension of RAM. It allows running larger programs than physically fit in memory, provides memory isolation/protection between processes, and enables more efficient multiprogramming.

Q55 What is demand paging?

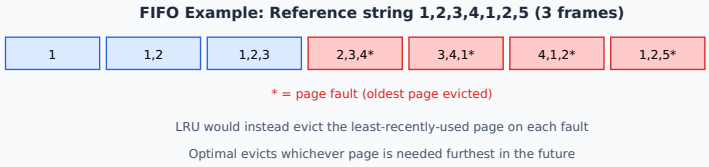
Demand paging is a virtual memory technique where a page is loaded into physical memory only when it is actually referenced/needed (on-demand), rather than loading the entire process into memory upfront — reducing initial load time and memory usage, since many pages of a program (e.g., rarely-used error-handling code) may never be needed during a given execution.

Q56 What is a page fault? What happens when one occurs?

A page fault occurs when a process tries to access a page that is not currently loaded in physical memory (marked invalid in the page table). When it occurs, the OS: 1) traps into the kernel, 2) checks if the access is valid, 3) finds a free frame (or evicts a page using a replacement algorithm if memory is full), 4) reads the required page from disk into that frame, 5) updates the page table, and 6) resumes the process's instruction.

Q57 What are common page replacement algorithms?

- **FIFO (First-In-First-Out):** replaces the oldest loaded page; simple but can suffer from Belady's anomaly.
- **Optimal (OPT):** replaces the page that won't be used for the longest time in the future; theoretically best but requires future knowledge, used only as a benchmark.
- **LRU (Least Recently Used):** replaces the page that hasn't been used for the longest time in the past — a practical approximation of OPT.
- **LFU (Least Frequently Used):** replaces the page accessed least often.
- **Clock/Second-Chance:** an efficient circular-buffer approximation of LRU using a reference bit.



Q58 What is Belady's Anomaly?

Belady's Anomaly is a counter-intuitive phenomenon (observed with the FIFO page replacement algorithm) where increasing the number of available page frames can actually result in *more* page faults for certain reference strings, rather than fewer as one might expect. LRU and Optimal algorithms do not suffer from this anomaly.

Q59 What is thrashing? How can it be resolved?

Thrashing occurs when a system spends most of its time swapping pages in and out of memory (servicing page faults) rather than executing actual process instructions, typically because too many processes are competing for too little physical memory. Resolution: reduce the degree of multiprogramming (fewer processes running at once), or ensure processes have their appropriate 'working set' of pages loaded (using the working-set model) before running.

Q60 What is the working set model?

The working set of a process is the set of pages it has referenced within a recent time window (Δ). The working-set model ensures a process is only allowed to run if all pages in its current working set can be kept resident in memory simultaneously — helping the OS decide the degree of multiprogramming and avoid thrashing.

Q61 **What is swapping in an OS?**

Swapping is moving an entire process (or significant chunks of it) between main memory and a disk-based swap space, temporarily freeing up RAM. It differs from paging/demand paging in granularity — classic swapping moves whole processes in/out, while paging operates at the level of individual fixed-size pages, often combined with swapping (i.e., swapping out individual pages rather than whole processes).

Storage, File Systems & I/O

11 Questions

Q62 What is a file system? What are its main functions?

A file system is the OS component responsible for organizing, storing, naming, retrieving, and managing data on storage devices, presenting files and directories in a structured, hierarchical way to users and applications, while managing free space and access permissions.

Q63 What are the different disk space allocation methods for files?

- **Contiguous allocation:** each file occupies a set of contiguous blocks — fast sequential/random access, but suffers from external fragmentation and difficulty growing files.
- **Linked allocation:** each file is a linked list of blocks scattered anywhere on disk — no external fragmentation, but slow random access (must traverse the list) and pointer overhead.
- **Indexed allocation:** a dedicated index block stores pointers to all of a file's blocks — supports efficient direct access without contiguous allocation's fragmentation issues (used in most modern file systems, e.g., inodes in Unix-like systems).

Q64 What is an inode in Unix-like file systems?

An inode (index node) is a data structure that stores metadata about a file — permissions, owner, size, timestamps, and pointers to the actual data blocks on disk — but notably NOT the file's name (names are stored separately in directory entries that map filenames to inode numbers).

Q65 What is the difference between a hard link and a symbolic (soft) link?

A **hard link** is a direct additional directory entry pointing to the same inode/data as the original file — both names are equally 'real'; deleting one doesn't affect the other as long as at least one link remains, but hard links can't span across different filesystems/partitions or usually link to directories. A **symbolic link** is a separate small file containing a path/reference to the target file's name — it can cross filesystems and link to directories, but breaks ('dangling link') if the target is moved/deleted.

Q66 What are the different disk scheduling algorithms?

- **FCFS:** services requests in arrival order; simple but can cause long seek times.
- **SSTF (Shortest Seek Time First):** services the request closest to the current head position next; can cause starvation for far-away requests.
- **SCAN (Elevator algorithm):** the disk arm moves in one direction servicing requests until it reaches the end, then reverses.
- **C-SCAN (Circular SCAN):** like SCAN, but after reaching one end, it jumps back to the beginning without servicing requests on the return trip, providing more uniform wait times.
- **LOOK / C-LOOK:** like SCAN/C-SCAN, but the arm only goes as far as the last request in each direction (not all the way to the disk's physical end), avoiding unnecessary travel.

SCAN/LOOK: head sweeps in one direction, servicing requests along the way

Q67 What is the difference between SCAN and C-SCAN disk scheduling?

SCAN moves the disk arm back and forth like an elevator, servicing requests in both directions. **C-SCAN** only services requests while moving in one direction; upon reaching the end, it quickly returns to the start without servicing requests along the way, then begins scanning again — this provides more uniform/predictable wait times across all disk positions since it doesn't 'double service' the middle tracks like SCAN does.

Q68 What is RAID? Name a few common RAID levels.

RAID (Redundant Array of Independent Disks) combines multiple physical disks into a logical unit for improved performance, redundancy, or both. Common levels: **RAID 0** (striping — improves performance, no redundancy), **RAID 1** (mirroring — full redundancy, halves usable capacity), **RAID 5** (striping with distributed parity — good balance of performance/redundancy, tolerates one disk failure), **RAID 10** (combination of mirroring and striping).

Q69 What is the difference between sequential access and direct (random) access to a file?

Sequential access reads/writes a file's data in order from the beginning, as with a tape (common for simple log files). **Direct (random) access** allows reading/writing at any arbitrary position within the file immediately, without needing to read through preceding data first — essential for database files and indexed data.

Q70 **What is buffering in the context of I/O?**

Buffering temporarily stores data in a memory area (buffer) while it's being transferred between a slower device (like a disk) and a faster one (like the CPU/application), helping smooth out speed mismatches between producer and consumer, reduce the number of actual I/O operations, and allow overlapping of computation with I/O.

Q71 **What is spooling in Operating Systems?**

Spooling (Simultaneous Peripheral Operations OnLine) is a technique where data destined for a slow device (like a printer) is temporarily stored in a queue/buffer on disk, allowing the requesting process to continue without waiting for the slow device, while the OS handles the actual output to the device separately, one job at a time, in the background.

Q72 **What is the difference between a block device and a character device?**

A **block device** (e.g., hard disks, SSDs) transfers data in fixed-size blocks and supports random access (can jump to any block). A **character device** (e.g., keyboards, mice, serial ports) transfers data as a continuous stream of individual characters/bytes, typically only supporting sequential access.

Advanced, Scenario-Based & Miscellaneous

13 Questions

Q73 What is the difference between a hard real-time and soft real-time system?

A **hard real-time system** must meet every deadline without exception — missing even one is considered a total system failure (e.g., anti-lock braking systems). A **soft real-time system** tries to meet deadlines but can tolerate occasional misses with a degraded (not catastrophic) outcome (e.g., a video call dropping a frame).

Q74 What is the difference between a microkernel and a hybrid kernel?

A **microkernel** keeps almost all services (drivers, file systems) outside the kernel in user space, communicating via message passing. A **hybrid kernel** (e.g., Windows NT kernel) is a compromise — it keeps some additional services in kernel space for performance reasons while retaining a somewhat modular structure, aiming to balance the performance of monolithic kernels with some of the modularity/stability benefits of microkernels.

Q75 What is copy-on-write (COW)? Where is it used?

Copy-on-write is an optimization where, when a resource (e.g., memory pages) is duplicated (such as when a process calls fork()), the copy isn't physically made immediately — both the original and 'copy' initially share the same physical memory pages (marked read-only). Only when either process attempts to *write* to a shared page does the OS actually create a separate physical copy for that process — saving time and memory if the data is never modified (very common with process forking in Unix-like systems).

Q76 What happens internally when you call fork() in Unix/Linux?

fork() creates a new child process that is (initially) an almost exact duplicate of the calling (parent) process — same code, data, open file descriptors, and (with copy-on-write) shared memory pages. It returns twice: 0 to the child process, and the child's actual PID to the parent, allowing the program to branch its behavior based on the return value.

Q77 What is the difference between exec() and fork() system calls?

`fork()` creates a new child process that is a copy of the calling process, continuing execution of the same program from the point right after the fork() call. `exec()` replaces the current process's memory image entirely with a new program, without creating a new process — the process ID stays the same, but the code/data/stack are completely replaced by the new program.

Q78 What is a system call vs a library function/API call?

A **system call** is a direct request to the kernel for a privileged operation (e.g., read(), write(), fork()), requiring a context switch into kernel mode. A **library function/API call** (e.g., printf()) runs entirely in user space and may internally invoke one or more system calls as needed (e.g., printf() eventually calls write()), but the function call itself doesn't necessarily require crossing into kernel mode every time.

Q79 What is DMA (Direct Memory Access)? Why is it used?

DMA allows peripheral devices (like disk controllers or network cards) to transfer data directly to/from main memory without requiring the CPU to manage every single byte of the transfer, freeing the CPU to perform other work concurrently while the transfer happens in the background — the CPU is only interrupted once the entire transfer is complete.

Q80 What is an interrupt? What is the difference between a hardware interrupt and a software interrupt (trap)?

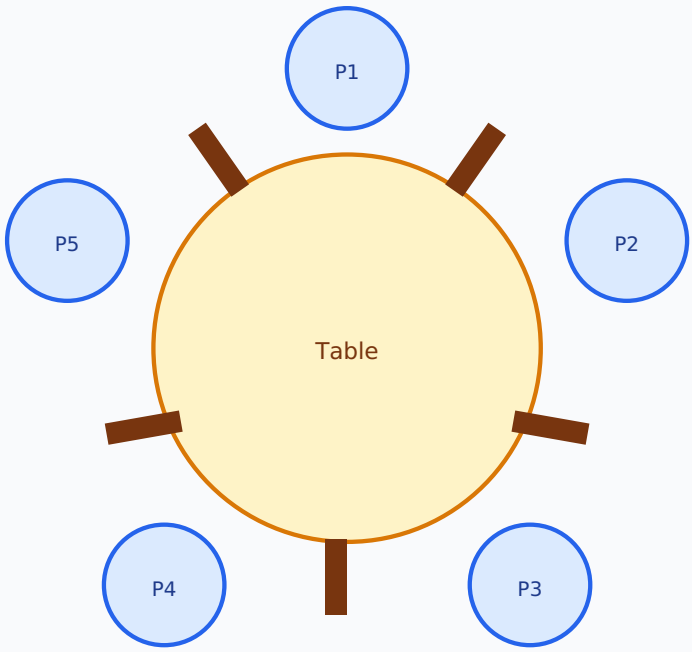
An interrupt is a signal that pauses the CPU's current execution to handle an urgent event, after which execution resumes where it left off. A **hardware interrupt** is triggered externally by a device (e.g., keyboard press, I/O completion) asynchronously to the running program. A **software interrupt (trap/exception)** is triggered internally by the currently executing program itself, either intentionally (a system call) or due to an error (e.g., divide by zero, invalid memory access).

Q81 What is priority inversion? How can it be resolved?

Priority inversion occurs when a higher-priority process/task is indirectly blocked from running because it's waiting on a resource (e.g., a lock) currently held by a lower-priority process, which itself gets preempted by medium-priority processes — effectively inverting the intended priority order. It's commonly resolved using **priority inheritance**, where the lower-priority process holding the lock temporarily inherits the higher priority of the process waiting on it, ensuring it finishes and releases the resource sooner.

Q82 **What is the dining philosophers problem, and what does it illustrate?**

It's a classic synchronization problem: five philosophers sit at a table with one fork between each pair, needing both adjacent forks to eat; if each simultaneously picks up their left fork and waits for their right, a deadlock (circular wait) occurs. It illustrates the challenges of avoiding deadlock and starvation when multiple processes compete for a limited set of shared resources, and is used to demonstrate/test solutions like resource ordering or limiting concurrent diners.



Each philosopher needs both forks — naive strategy can deadlock

Q83 **What is the producer-consumer (bounded buffer) problem?**

It's a classic synchronization problem where a 'producer' process generates data items and places them in a shared, fixed-size buffer, while a 'consumer' process removes and processes them — requiring synchronization to ensure the producer doesn't add to a full buffer, the consumer doesn't remove from an empty buffer, and both don't corrupt the buffer by accessing it simultaneously. It's typically solved using semaphores (empty/full counting semaphores plus a mutex).

Q84 **What is a livelock, and how is it different from a deadlock?**

In a **deadlock**, processes are blocked and make no progress at all. In a **livelock**, processes are not blocked — they keep actively changing their states/responding to each other (e.g., repeatedly trying to be polite and yielding to one another), but none of them make any actual forward progress toward completing their tasks, similar to two people repeatedly stepping aside for each other in a hallway and never getting past.

Q85 **How would you debug a system that appears to be hanging (unresponsive)?**

Check whether it's a true deadlock or just heavy load: inspect running processes/threads and their states (e.g., using tools like ps, top, or thread dumps), look for processes stuck waiting on a lock/resource for an unusually long time, check for circular wait patterns among held/requested resources, and review recent logs for errors or resource exhaustion (e.g., out of memory, too many open files) that might explain the stall.

Quick Interview Tips

- Screening rounds usually start with process vs thread, process states, and basic scheduling algorithms.
- Coding/online rounds sometimes ask you to simulate scheduling algorithms (compute waiting/turnaround time) or solve producer-consumer with semaphores — practice these by hand.
- Technical panel rounds go deeper into paging vs segmentation, deadlock conditions, and page replacement algorithms — be ready to trace through examples on a whiteboard.
- Senior rounds may explore thrashing, priority inversion, and real production debugging scenarios (hangs, high CPU, memory leaks).
- Whenever possible, sketch a quick diagram (process state diagram, RAG, Gantt chart) while explaining — it signals real understanding, not memorization.
- Practice explaining answers out loud in under 60–90 seconds each; interviewers value clarity over length.